

Цель работы

Ознакомиться с методами и средствами построения отказоустойчивых решений на базе СУБД Postgres; получить практические навыки восстановления работы системы после отката.

Требования к выполнению работы

- В качестве хостов использовать одинаковые виртуальные машины.
- В первую очередь необходимо обеспечить сетевую связность между ВМ.
- Для подключения к СУБД, например через `psql`, использовать отдельную виртуальную или физическую машину.
- Демонстрировать наполнение базы и доступ на запись на примере не менее чем двух таблиц, столбцов, строк, транзакций и клиентских сессий.

Этапы выполнения работы

Этап 1. Конфигурация

Развернуть Postgres на двух узлах в режиме горячего резерва: Master + Hot Standby. Не использовать дополнительные пакеты. Продемонстрировать доступ в режиме чтения и записи на основном сервере, доступ в режиме чтения на резервном сервере, а также актуальность данных на нём.

Для развёртывания нескольких узлов Postgres и обеспечения сетевой связности использовался *docker compose*.

docker-compose.yml

Два контейнера находятся в общей сети `pg-net`.

```
services:
  pg-1:
    image: postgres:16
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: secret123
    ports:
      - "5432:5432"
    volumes:
      - ./pgdata/pg1:/var/lib/postgresql/data
    networks:
      - pg-net

  pg-2:
    image: postgres:16
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: secret123
    ports:
      - "5433:5432"
    volumes:
```

```

- ./pgdata/pg2:/var/lib/postgresql/data
networks:
- pg-net

networks:
pg-net:
driver: bridge
name: pg-net

```

Шаги для режима Hot Standby

pg-1 — основной узел.

pg-2 — резервный узел.

На pg-1 в файл `postgresql.conf` были добавлены следующие параметры:

```

listen_addresses = '*'
max_connections = 100
shared_buffers = 128MB
dynamic_shared_memory_type = posix

# hot-standby setup
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
wal_keep_size = 256MB
hot_standby = on

max_wal_size = 1GB
min_wal_size = 80MB
log_timezone = 'Etc/UTC'
datestyle = 'iso, mdy'
timezone = 'Etc/UTC'
lc_messages = 'en_US.utf8'
lc_monetary = 'en_US.utf8'
lc_numeric = 'en_US.utf8'
lc_time = 'en_US.utf8'
default_text_search_config = 'pg_catalog.english'

```

В файл `pg_hba.conf` были добавлены правила доступа:

```

host replication replicator 0.0.0.0/0 md5
host all all 0.0.0.0/0 md5

```

На pg-1 был создан пользователь для репликации:

```

CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'replpass';

```

Создать backup pg-1 и положить его в pg-2

```
docker run --rm \
  --network pg-net \
  -v "$PWD/pgdata/pg2:/var/lib/postgresql/data" \
  postgres:16 \
  bash -c '
    export PGPASSWORD=replpass
    pg_basebackup \
      -h rshd-lab-3-pg-1-1 \
      -p 5432 \
      -U replicator \
      -D /var/lib/postgresql/data \
      -Fp \
      -Xs \
      -P
  '
```

На pg-2 добавить в postgresql.conf

```
hot_standby = on
primary_conninfo = 'host=rshd-lab-3-pg-1-1 port=5432 user=replicator
password=replpass application_name=pg2'
```

На pg-2 создать standby signal

```
touch ./pgdata/pg2/standby.signal
sudo chown -R 999:999 ./pgdata/pg2
```

Проверка репликации

Проверка pg_stat_replication

```
postgres=# SELECT application_name, state, sync_state, client_addr
FROM pg_stat_replication;
 application_name | state | sync_state | client_addr
-----+-----+-----+-----
pg2               | streaming | async      | 172.21.0.2
(1 row)
```

На основном сервере была создана тестовая таблица:

```
CREATE TABLE repl_test(
  id serial primary key,
  msg text
);

INSERT INTO repl_test(msg)
VALUES ('hello from master');
```

На резервном сервере была выполнена проверка:

```
SELECT * FROM repl_test;
```

id	msg
-----+	-----
1	hello from master